

Design Autocomplete

Google · Amazon · Flipkart · Swiggy – search box lo ప్రతి అక్షరం ఒక ప్రశ్న, మరియు జవాబు **50 ms లోపల** రావాలి.

నిజమైన 2,34,428-పదాల నిఘంటువుతో కొలిచినవి. మొదటి విరుపు "hel" కి **helbeh, helcoid, helcology** అని సూచిస్తుంది. రెండోది ఒక్కో అక్షరానికీ **2.34** లక్షల పదాలని తాకుతుంది. మూడోది ఒక పదం **trending** అయినా గమనించదు.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఈ doc lo ఒక తేడా ఉంది – ఇక్కడి డేటా కృత్రిమమైనది కాదు.

నిజమైన నిఘంటువు

ప్రతి macOS మరియు Linux యంత్రంలో `/usr/share/dict/words` అనే file ఉంటుంది. నా దగ్గర అది **2,35,976 పదాలు**. Lowercase చేసి, రెండు అక్షరాల కంటే తక్కువవి తీసేసి, **నకిలీలు తొలగించాక 2,34,428** మిగిలాయి.

(ఆ నకిలీల కథ §4 lo ఉంది – అవి ఒక నిజమైన bug ని బయటపెట్టాయి.)

ప్రజాదరణ మాత్రం కృత్రిమమైనది – నా దగ్గర Google శోధన గణాంకాలు లేవు, కాబట్టి సాధారణ ఆంగ్ల పదాలకి ఎక్కువ, మిగతావాటికి Zipf తోక ఇచ్చాను. అది ఒక ఊహ, మరియు దాన్ని ఇక్కడ స్పష్టంగా చెప్తున్నాను.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ రెండు ప్రశ్నలు ఉన్నాయి
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: తప్పు ఐదు

3. Step – `filter` + `slice` · పదిహేను అక్షరాలు
4. మొదటి విరుపు – "hel" → helbeh, helcoid, helcology
5. Step – ప్రజాదరణ ప్రకారం క్రమబద్ధీకరణ

Part 3 – రెండో విరుపు: ప్రతి అక్షరానికీ మొత్తం నిఘంటువు

6. Step – కొలిచి చూద్దాం
7. రెండో విరుపు – 5 సూచనలకి 2,34,428 పదాలు
8. Step – Trie · prefix ని నడవడం, వెతకడం కాదు

Part 4 – మూడో విరుపు: దాచిన జవాబు పాతబడింది

9. Step – ప్రతి node వద్దా top-5 ని దాచడం
10. మూడో విరుపు – helicopter trending అయింది, ఎవరూ గమనించలేదు
11. Step – మారిన దారిని మాత్రమే మళ్ళీ కట్టడం

Part 5 – పూర్తి system

12. Step – పరీక్ష ఒక bug ని దాచింది
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీతో చెప్పాల్సిన English script
16. Follow-ups – అక్షర దోషాలు, వ్యక్తిగతీకరణ, scale
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ రెండు ప్రశ్నలు ఉన్నాయి

"Design a search autocomplete system. As the user types, show the top suggestions for what they've typed so far."

ఇందులో రెండు వేర్వేరు ప్రశ్నలు దాగి ఉన్నాయి, మరియు చాలా మంది రెండోదాన్ని అస్సలు చూడరు:

"TOP SUGGESTIONS" – రెండు పదాలు, రెండు PROBLEMS

ఒకటి వేగం గురించి • రెండోది నాణ్యత గురించి

"suggestions" – వేగం

ప్రతి keystroke ఒక శోధన
"hello" అంటే ఐదు శోధనలు
ప్రతిది 50 ms లోపల
→ §7 • 2.29 ms x QPS

"top" – నాణ్యత

ఏ ఐదు? ఎందుకు ఆ ఐదు?
ఇది ఒక ర్యాంకింగ్ problem –
వెతకడం problem కాదు
→ §4 • helbeh, helcoid

మరియు అవి ఒకదానికొకటి ఎదురుగా ఉంటాయి

ర్యాంకింగ్ ఖరీదు పంచుతుంది – ఐదు ఇవ్వాలంటే ముందు అన్నిటిని క్రమబద్ధీకరించాలి.
వేగం కోసం జవాబులని దాచితే – ప్రజాదరణ మారినప్పుడు అవి పాతబడతాయి (§10).
కాబట్టి ఈ problem యొక్క నిజమైన ఆకారం: వేగం ↔ తాజాదనం అనే వ్యాపారం.
Interview Io ఆ వ్యాపారాన్ని పేరు పెట్టి చెప్పడమే నగం జవాబు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఎన్ని సూచనలు? (5? 10?)	K చిన్నది అంటే ప్రతి node వద్దా దాచొచ్చు (§9)

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
సూచనలని దేని ప్రకారం క్రమబద్ధీకరించాలి?	ఇదే §4 – "top" అంటే ఏమిటో నిర్వచించకపోతే జవాబు అర్థరహితం
ఎన్ని పదాలు / queries?	§7 – లక్షల్లో అంటే scan పనికిరాదు
ప్రజాదరణ ఎంత తరచుగా మారుతుంది?	అత్యంత ముఖ్యమైన ప్రశ్న – §10 lo cache నిర్ణయాన్ని ఇదే నిర్ణయిస్తుంది
అక్షర దోషాలు క్షమించాలా? ("gogle" → "google")	అవును అంటే పూర్తిగా వేరే structure (§16)
ఒక్కో user కి వేరే సూచనలా?	అవును అంటే వ్యక్తిగత layer (§16)
మధ్యలో సరిపోయే పదాలు? ("book" → "facebook")	అవును అంటే trie సరిపోదు (§16)

రెండో ప్రశ్న అడగకపోతే మే జవాబు తప్పు – మరియు అది మేకు తెలియదు

"సూచనలని దేని ప్రకారం క్రమబద్ధీకరించాలి?"

చాలా మంది దీన్ని అడగరు, ఎందుకంటే "top 5" అనేది స్పష్టంగా అనిపిస్తుంది. వారు `filter(...).slice(0, 5)` రాస్తారు.

కానీ `slice` ఇచ్చేది మొదటి ఐదు – ఉత్తమ ఐదు కాదు. నిఘంటువు అక్షర క్రమంలో ఉంటే, "hel" అని type చేస్తే మీకు **helbeh, helcoid, helcology** వస్తాయి.

Code lo ఏ bug లేదు. Test pass అవుతుంది. ఉత్పత్తి మాత్రం పనికిరానిది.

Part 2 – మొదటి విరుపు: తప్పు ఐదు

3. Step – filter + slice · పదిహేను అక్షరాలు

```
class Autocomplete {
  constructor(words = []) { this.words = words; }
  add(w) { this.words.push(w); }
  suggest(prefix, k = 5) {
    return this.words
      .filter(w => w.startsWith(prefix))
      .slice(0, k);
  }
}
```

చిన్న ఉదాహరణతో ఇది పరిపూర్ణంగా పనిచేస్తుంది:

```
"ca" → [ 'cat', 'car', 'card', 'care', 'careful' ]
"car" → [ 'car', 'card', 'care', 'careful', 'cart' ]
"do" → [ 'dog', 'do', 'door' ]
"z" → [ ]
```

ఇప్పుడు నిజమైన నిఘంటువు – 2,34,428 పదాలు – పెట్టి చూద్దాం.

4. మొదటి విరుపు – "hel" → helbeh, helcoid, helcology

```
"car"
  మొదటి 5 : car, cara, carabao, carabeen, carabid
"fa"
  మొదటి 5 : fa, faba, fabaceae, fabaceous, fabella
"hel"
  మొదటి 5 : helbeh, helcoid, helcology, helcoplasty, helcosis
"ind"
  మొదటి 5 : ind, indaba, indaconitine, indagate, indagation
```

మొదటి విరుపు - "HELLO" అనే పదమే జాబితాలో లేదు

User "hel" అని టైప్ చేశాడు. అతనికి కావాల్సినది స్పష్టంగా **hello, help, held**.

System ఇచ్చినది: **helbeh, helcoid, helcology, helcoplasty, helcosis** – ఇవి అన్నీ నిజమైన పదాలే, కానీ ఏ ఒక్కరూ వీటిని వెతకరు.

మౌలిక తప్పు: `slice(0, 5)` అనేది "ఉత్తమ ఐదు" కాదు – అది "నిఘంటువులో ముందు కనిపించిన ఐదు". మరియు నిఘంటువు **అక్షర క్రమంలో** ఉంది, కాబట్టి జవాబు ఎప్పుడూ **అక్షరక్రమంలో మొదటివి** – అంటే దాదాపు ఎప్పుడూ అరుదైన పదాలు.

ఇది ఒక performance bug కాదు, ఒక crash కాదు. ఇది నిశ్శబ్దంగా పనికిరాని ఉత్పత్తి.

మరియు ఒక చిన్న కథ – నకిలీలు

నిఘంటువుని lowercase చేశాక ఒక విషయం గమనించాను:

మొడి పదాలు	: 2,35,976
lowercase తర్వాత:	2,35,922
ప్రత్యేకమైనవి	: 2,34,428
నకిలీలు	: 1,494
ఉదాహరణలు	: abelite, abigail, abu, academic, acom, ...

1,494 నకిలీలు – "Abigail" మరియు "abigail" రెండూ ఉన్నాయి, lowercase చేశాక ఒకటే.

నేను దీన్ని ఎలా కనుగొన్నాను? – **trie ని scan తో పోల్చినప్పుడు**. Trie `part` అని ఇచ్చింది, scan `part`, `part` అని ఇచ్చింది. మొదట trie lo bug ఉందనుకున్నాను. నిజానికి **scan ఒకే సూచనని రెండుసార్లు** ఇస్తోంది – ఎందుకంటే డేటాలో ఆ పదం రెండుసార్లు ఉంది.

Trie సహజంగానే నకిలీలని తొలగిస్తుంది (ఒక పదం = ఒక node). Scan తొలగించదు. ఇది **naive version యొక్క మూడో లోపం** – మరియు ఇది **డేటా వల్ల** వచ్చింది, code వల్ల కాదు.

5. Step – ప్రజాదరణ ప్రకారం క్రమబద్ధీకరణ

"Top" అంటే ఏమిటో **నిర్వచించాలి**. Autocomplete lo అది దాదాపు ఎప్పుడూ **శోధన సంఖ్య**:

```
const best5 = (prefix, k = 5) =>
  words.filter(w => w.startsWith(prefix))
    // ప్రజాదరణ, ఆపై అక్షరక్రమం
    .sort((a, b) => count(b) - count(a) || a.localeCompare(b))
    .slice(0, k);
```

ఆ `|| a.localeCompare(b)` ముఖ్యం – **సమాన ప్రజాదరణ ఉన్నవాటికి ఒక నిర్ణయాత్మక క్రమం**. లేకపోతే సూచనలు కారణం లేకుండా మారుతూ ఉంటాయి (Deep Dive 13 §10 lo చూసిన అదే సమస్య).

"car"

మొదటి 5 : car, cara, carabao, carabeen, carabid

సరైన 5 : car, card, care, careful, carry

"fa"

మొదటి 5 : fa, faba, fabaceae, fabaceous, fabella

సరైన 5 : father, family, face, fact, factory

"hel"

మొదటి 5 : helbeh, helcoid, helcology, helcoplasty, helcosis

సరైన 5 : hello, help, helbeh, helcosis, helepole

"ind"

మొదటి 5 : ind, indaba, indaconitine, indagate, indagation

సరైన 5 : india, index, indian, ind, indagatory

ఇప్పుడు జవాబులు అర్థవంతంగా ఉన్నాయి. కానీ...

Part 3 – రెండో విరుపు: ప్రతి అక్షరానికి మొత్తం నిఘంటువు

6. Step – కొలిచి చూద్దాం

ఒక ముఖ్యమైన వివరం: **autocomplete ప్రతి keystroke కీ నడుస్తుంది**. User "helicopter" అని టైప్ చేస్తే అది **పది శోధనలు**, ఒక్కటి కాదు.

200 యాదృచ్ఛిక పదాలని అక్షరం అక్షరంగా టైప్ చేద్దాం – అది **1,515 keystrokes**:

```
const typed = [];  
for (let i = 0; i < 200; i++) {  
  const w = ALL[Math.floor(rand() * ALL.length)];  
  for (let n = 1; n <= Math.min(w.length, 8); n++) typed.push(w.slice(0, n));  
}
```

7. రెండో విరుపు – 5 సూచనలకి 2,34,428 పదాలు

1,515 keystrokes
ranked scan : 3463 ms → 2.29 ms/keystroke
ఒక్కే keystroke కి తాకిన పదాలు: 2,34,428

prefix పొడవు	సరిపోయే పదాలు (సగటున)
1	15,025
2	3,703
3	597
4	143
5	62

ఒక్కే KEYSTROKE • జవాబు 5, పని 2.34 లక్షలు

ప్రతి అక్షరానికి మొత్తం నిఘంటువుని చదువుతున్నాం

2,34,428 పదాలని తాకాం

15,025 సరిపోయాయి (ఒక అక్షరం prefix కి)

5 చూపించాం

ఒక్క user కి **2.29 ms** – అది భరించదగినది.

కానీ ఒక search engine సెకనుకి **10,000 queries** చూస్తుంది.

$2.29 \text{ ms} \times 10,000 = \text{సెకనుకి 23 సెకన్ల CPU}$. అంటే 23 servers, ఒక్క search box కోసం.

మరియు నిఘంటువు పెరిగితే ఆ సంఖ్య నేరుగా పెరుగుతుంది.

రెండో విరుపు - PREFIX ఒక FILTER అయింది, ఒక దారి కాలేదు

User "h" అని టైప్ చేశాడు. System **2,34,428 పదాలని** చదివి, ప్రతిదాన్నీ "h తో మొదలవుతుందా?" అని అడిగింది. అందులో 15,025 అవునన్నాయి, వాటిని క్రమబద్ధీకరించి **ఐదు** చూపించింది.

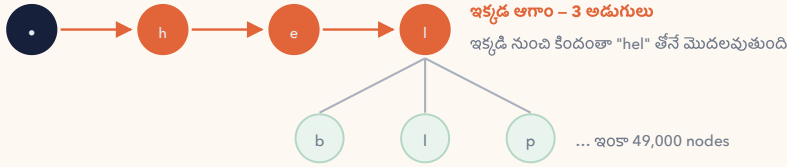
మౌలిక తప్పు: "hel" అనే prefix ఒక **దారి** - h, ఆపై e, ఆపై l. ఆ నిర్మాణం డేటాలో ఉంది, కానీ మనం దాన్ని **వాడట్లేదు** - ప్రతిసారీ ప్రతి పదాన్నీ మళ్ళీ చదువుతున్నాం.

Deep Dive 12 §4 lo సరిగ్గా ఇదే ఆకారం చూశాం: path ని ఒక string గా చూస్తే, దానిలోని నిర్మాణాన్ని ప్రతిసారీ తిరిగి కనుక్కోవాలి.

8. Step - Trie · prefix ని నడవడం, వెతకడం కాదు

TRIE · ప్రతి అక్షరం ఒక అడుగు

"hel" కి చేరడం = మూడు అడుగులు, 2,34,428 పోలికలు కాదు



Node ని కనుక్కోవడం **3 అడుగులు**. కానీ ఉత్తమ ఐదు తెలియాలంటే -

ఆ node కింద ఉన్న **ప్రతి పదాన్నీ** చూసి క్రమబద్ధీకరించాలి.

ఒక అక్షరం prefix కి అది **49,107 nodes**. సమస్య సగమే పరిష్కారమైంది.

```
class TrieNode {
  constructor() { this.kids = new Map(); this.word = null; this.count = 0; }
}
class Trie {
  constructor() { this.root = new TrieNode(); this.nodes = 1; }
  insert(word, count = 1) {
    let cur = this.root;
    for (const ch of word) {
      if (!cur.kids.has(ch)) { cur.kids.set(ch, new TrieNode()); this.nodes++; }
      cur = cur.kids.get(ch);
    }
    cur.word = word; cur.count = count;
  }
  nodeFor(prefix) { // 0(prefix పొడవు)
    let cur = this.root;
    for (const ch of prefix) {
      cur = cur.kids.get(ch);
      if (!cur) return null;
    }
    return cur;
  }
  // ఆ node కింద ఉన్న పదాలని సేకరించి, ప్రజాదరణ ప్రకారం top-k
  suggest(prefix, k = 5) {
    const start = this.nodeFor(prefix);
```

```

    if (!start) return { words: [], visited: 0 };
    const found = []; let visited = 0;
    const walk = (n) => {
        visited++;
        if (n.word) found.push([n.word, n.count]);
        for (const c of n.kids.values()) walk(c);
    };
    walk(start);
    found.sort((a,b) => b[1]-a[1] || a[0].localeCompare(b[0]));
    return { words: found.slice(0,k).map(x=>x[0]), visited };
}
}

```

plain trie (7,58,880 nodes)
691 ms → 0.46 ms/keystroke · 8,550 nodes/keystroke

prefix పొడవు	చూసిన nodes (సగటున)
1	49,107
2	12,635
3	2,191
4	529
5	249
6	44

2.29 ms → 0.46 ms, ఐదు రెట్లు వేగం. కానీ ఆ మొదటి వరుస చూడండి: **ఒక అక్షరం prefix కి 49,107 nodes**.

సగం పరిష్కారం - మరియు మిగిలిన సగమే ముఖ్యమైనది

Trie **node** ని కనుక్కోవడాన్ని పరిష్కరించింది - మూడు అక్షరాల prefix కి మూడు అడుగులు.

కానీ **ఉత్తమ ఐదు ఏవో తెలుసుకోవడాన్ని** అది పరిష్కరించలేదు. ఆ node కింద ఉన్నవన్నీ చూడాలి.

మరియు ఇది అతి ముఖ్యమైన చోట చెడ్డది: user టైప్ చేసే **మొదటి అక్షరం** - అప్పుడే అతను ఎక్కువగా ఎదురుచూస్తాడు, మరియు అప్పుడే మనం 49,107 nodes నడుస్తున్నాం. Prefix పొడవు పెరిగితే ఖర్చు తగ్గుతుంది - అంటే **ఖర్చు సరిగ్గా తిరగబడి ఉంది**.

Part 4 – మూడో విరుపు: దాచిన జవాబు పాతబడింది

9. Step – ప్రతి node వద్దా top-5 ని దాచడం

ఇక్కడ ఒక కీలకమైన గమనిక: **K చిన్నది (5)**, మరియు **ఒక node యొక్క top-5 దాని పిల్లల top-5 ల నుంచి మాత్రమే రాగలదు**.

ఎందుకు? – ఒక పదం ఒక child యొక్క subtree లో 6వ స్థానంలో ఉంటే, **అదే subtree లో దాన్ని ఓడించే ఐదు పదాలు ఉన్నాయి**. ఆ ఐదు parent యొక్క subtree లో కూడా ఉన్నాయి. కాబట్టి ఆ 6వ పదం parent యొక్క top-5 లో **ఎప్పటికీ ఉండదు**.

అంటే top-5 ని **కింది నుంచి పైకి ఒకసారి కట్టొచ్చు**, ప్రతి node వద్దా కేవలం ఐదు entries నిల్వ చేస్తూ:

```
// రెండు క్రమబద్ధమైన top-K జాబితాలని కలిపి, మళ్ళీ top-K
function merge(a, b, k = K) {
  const out = []; let i = 0, j = 0;
  while (out.length < k && (i < a.length || j < b.length)) {
    if (j >= b.length) out.push(a[i++]);
    else if (i >= a.length) out.push(b[j++]);
    else {
      const [wa, ca] = a[i], [wb, cb] = b[j];
      out.push(cb > ca || (cb === ca && wb < wa) ? b[j++] : a[i++]);
    }
  }
  return out;
}

#recompute(n) {
  let top = n.word ? [[n.word, n.count]] : [];
  for (const kid of n.kids.values()) top = merge(top, kid.top, this.k);
  n.top = top;
}
```

suggest ఇప్పుడు **ఒక నడక, ఒక slice**:

```
suggest(prefix, k = this.k) {
  let cur = this.#root;
  for (const ch of prefix) {
    cur = cur.kids.get(ch);
    if (!cur) return [];
  }
  return cur.top.slice(0, k).map(([w]) => w);
}
```

0.46 ms → 0.2 μs. ఒక్క keystroke కి 4.4 nodes మాత్రమే – prefix పొడవు అంతే.

10. మూడో విరుపు – helicopter trending అయింది, ఎవరూ గమనించలేదు

ఒక పదం ఒక్కసారిగా ప్రజాదరణ పొందింది:

```
const n = nodeFor('helicopter');  
n.count = 9_000_000; // జాబితాలో అందరికంటే ఎక్కువ
```

```
ఇప్పుడు "hel" సూచనలు:  
hello, help, helbeh, helcosis, helepole  
  
--- "helicopter" ఒక్కసారిగా trending అయింది ---  
పాత count : 3  
కొత్త count: 90,00,000 (జాబితాలో అందరికంటే ఎక్కువ)  
  
"hel" సూచనలు ఇప్పుడు:  
hello, help, helbeh, helcosis, helepole  
"helicopter" ఉందా? లేదు x  
  
"helicopte" (ఆ node కి సరిగ్గా పైన): helicopter
```

మూడో విరుపు – దాచిన జవాబు సమయంలో గడ్డకట్టుకుపోయింది

"helicopter" ఇప్పుడు మొత్తం నిఘంటువులో అత్యధికంగా వెతికే పదం. కానీ "hel" అని టైప్ చేస్తే అది కనిపించదు.

మరియు చివరి పంక్తి చూడండి: **"helicopte" కి అది కనిపిస్తోంది** – ఎందుకంటే ఆ node యొక్క top-5 lo దాని సొంత పదం ఉంది. కానీ దాని పూర్వీకులు ఎవరూ తెలుసుకోలేదు.

మౌలిక తప్పు: మనం ఒక ఉత్పన్న విలువని నిల్వ చేశాం (top-5), కానీ దాని మూలం మారినప్పుడు దాన్ని సరిచేసే మార్గం రాయలేదు. Cache అంటే అదే – మరియు ప్రతి cache కి ఒక **invalidation** కథ ఉండాలి.

Deep Dive 15 §16 lo directory size cache గురించి ఇదే చెప్పాం: "cache ఒక సులభమైన జవాబులా కనిపిస్తుంది, కానీ అది ఒక కొత్త స్థిరత్వ సమస్యని పుట్టిస్తుంది." ఇక్కడ అది నిజంగా జరిగింది.

11. Step – మారిన దారిని మాత్రమే మళ్ళీ కట్టడం

పరిష్కారం సూటిది: ఒక పదం count మారితే, **root నుంచి ఆ పదం వరకు ఉన్న దారిలోని nodes మాత్రమే మారగలవు.** వాటిని కింది నుంచి పైకి మళ్ళీ కట్టడం:

```

bump(word, delta = 1) {
  const path = this.#path(word, false);
  if (!path) return { ok: false, reason: `NOT_FOUND: ${word}` };
  const leaf = path[path.length - 1];
  if (leaf.word === null) return { ok: false, reason: `NOT_A_WORD: ${word}` };
  leaf.count += delta;
  for (let i = path.length - 1; i >= 0; i--) this.#recompute(path[i]);
  return { ok: true, count: leaf.count };
}

```

ఖర్చు: దారి పొడవు $\times$ (పిల్లల సంఖ్య $\times$ K). "helicopter" కి అది 10 nodes – మొత్తం 7,58,880 కాదు.

```

helicopter trending → { ok: true, count: 9000003 }
"hel" ఇప్పుడు → helicopter, hello, help, helbeh, helcosis
"h" ఇప్పుడు → helicopter, he, his, have, had

```

"h" వద్ద కూడా కనిపిస్తోంది – దారి మొత్తం root వరకు సరిచేయబడింది.

మరియు ఇక్కడ ఒక వ్యాపారం ఉంది – దాన్ని పేరు పెట్టి చెప్పండి

ఇప్పుడు **ప్రతి insert కూడా** దారిని మళ్ళీ కడుతుంది. మొదటి loading lo అది ఖరీదు:

ఒక్కొక్కటిగా insert : 851–929 ms

loadAll (bulk) : 194–291 ms → 3–4× వేగం

అందుకే **రెండు దారులు** ఉన్నాయి: **loadAll** అన్నీ చేర్చి **చివర్లో ఒకసారి** కడుతుంది (మొదటి loading కి), మరియు **insert / bump** దారిని **వెంటనే** సరిచేస్తాయి (నడుస్తున్న system కి).

ఇది ఒక సాధారణ నమూనా: bulk load మరియు incremental update వేర్వేరు మార్గాలు. ఒక్కటే రాస్తే, ఏదో ఒకటి తప్పకుండా నెమ్మదవుతుంది.

Part 5 – పూర్తి system

12. Step – పరీక్ష ఒక bug ని దాచింది

bump, insert, remove – మూడూ cache ని సరిచేయాలి. దాన్ని ఎలా నమ్మాలి? – **fuzz test**: 2,000 యాదృచ్ఛిక మార్పుల తర్వాత, trie యొక్క ప్రతి జవాబునీ ఒక సాదా reference తో పోల్చడం.

మొదటిసారి నడిపినప్పుడు **12 తేడాలు** వచ్చాయి:

```
"d"
  trie: do|down|day|did|dogmatism
  scan: do|down|day|did|dracocephalum
```

నేను వెంటనే merge lo bug ఉందని అనుకున్నాను. కానీ ఆ ఖచ్చితమైన operation ని వెతికితే:

```
op #1182: [ 'bump', 'dracocephalum', 982706 ]
```

```
// నా test lo ఉన్నది:
ac.bump(w, d); // ← తిరిగి వచ్చిన విలువని చూడలేదు
live.set(w, (live.get(w) ?? 0) + d);
```

BUG CODE LO లేదు – అది పరీక్షలో ఉంది

ఆ పదం కొన్ని operations ముందు **తొలగించబడింది**. కాబట్టి:

```
bump ఉన్న పదం : { ok: true, count: 150 }
తీసేశాక bump : { ok: false, reason: 'NOT_FOUND: dracocephalum' }
```

bump సరిగ్గానే తిరస్కరించింది. కానీ నా పరీక్ష ఆ తిరస్కరణని **విస్మరించి**, reference lo ఆ పదాన్ని తిరిగి చేర్చింది. అప్పటినుంచి reference తప్పు, trie సరైనది – మరియు పరీక్ష **trie ని నిందించింది**.

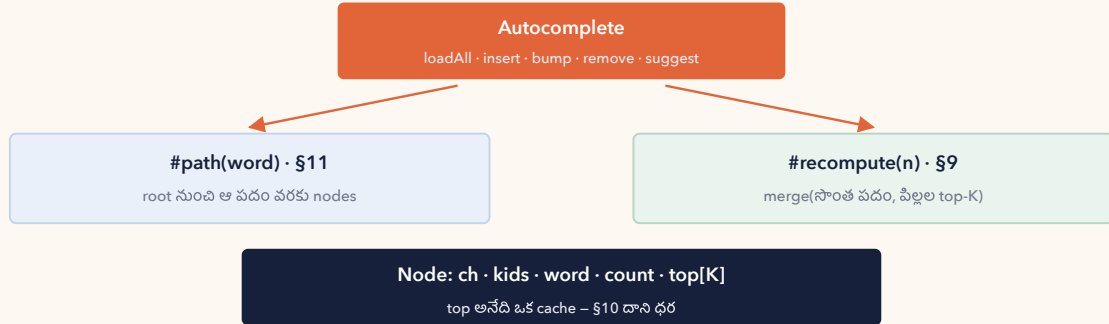
పాఠం: ఒక fuzz test ఒక operation యొక్క తిరిగి వచ్చిన విలువని విస్మరిస్తే, అది ఒక వేరే system ని పరీక్షిస్తోంది. మరియు అది విఫలమైనప్పుడు, **మీ మొదటి అనుమానం తప్పు చోట** ఉంటుంది.

సరిచేశాక: **1,998 మార్పులు · 1,494 prefixes · సరైనది ✓**

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · ప్రతి **NODE** ఒక జవాబుని పట్టుకుంటుంది

Node lo చూడు విషయాలు - అక్షరం, పదం (ఉంటే), మరియు దానిన top-K



```
'use strict';
const K = 5;

class Node {
  constructor(ch) {
    this.ch = ch; this.kids = new Map();
    this.word = null; this.count = 0;
    this.top = []; // [[word, count], ...] - గరిష్ఠం K
  }
}

// రెండు క్రమబద్ధమైన top-K జాబితాలని కలిపి, మళ్ళీ top-K
function merge(a, b, k = K) {
  const out = []; let i = 0, j = 0;
  while (out.length < k && (i < a.length || j < b.length)) {
    if (j >= b.length) out.push(a[i++]);
    else if (i >= a.length) out.push(b[j++]);
    else {
      const [wa, ca] = a[i], [wb, cb] = b[j];
      out.push(cb > ca || (cb === ca && wb < wa) ? b[j++] : a[i++]);
    }
  }
  return out;
}

class Autocomplete {
  #root = new Node('');
  #nodes = 1;
  constructor(k = K) { this.k = k; }
  get size() { return this.#nodes; }

  // ---- ఒక పదం దారి; create=true అయితే లేని nodes తయారు చేస్తుంది ----
  #path(word, create = false) {
    const path = [this.#root];
    let cur = this.#root;
    for (const ch of word) {
      let next = cur.kids.get(ch);
      if (!next) {
        if (!create) return null;
        next = new Node(ch); cur.kids.set(ch, next); this.#nodes++;
      }
    }
  }
```

```

    path.push(next); cur = next;
  }
  return path;
}
// ఒక node యొక్క top-K ని దాని పిల్లల నుంచి మళ్ళీ కట్టడం
#recompute(n) {
  let top = n.word ? [[n.word, n.count]] : [];
  for (const kid of n.kids.values()) top = merge(top, kid.top, this.k);
  n.top = top;
}

// ఒక్కొక్కటిగా చేర్చడం - దారిలోని top-K వెంటనే సరిచేయబడుతుంది
insert(word, count = 1) {
  if (!word) return { ok: false, reason: 'EMPTY_WORD' };
  const path = this.#path(word, true);
  const leaf = path[path.length - 1];
  leaf.word = word; leaf.count = count;
  for (let i = path.length - 1; i >= 0; i--) this.#recompute(path[i]);
  return { ok: true };
}

// మొదటి loading కి - అన్నీ చేర్చి, ఆపై ఒకసారి కింది నుంచి పైకి
loadAll(entries) {
  for (const [word, count] of entries) {
    if (!word) continue;
    const path = this.#path(word, true);
    const leaf = path[path.length - 1];
    leaf.word = word; leaf.count = count ?? 1;
  }
  const build = (n) => {
    for (const kid of n.kids.values()) build(kid);
    this.#recompute(n);
  };
  build(this.#root);
  return { ok: true, words: entries.length, nodes: this.#nodes };
}

// ---- ఒక పదం ప్రజాదరణ మారింది - పూర్వీకులని మాత్రమే మళ్ళీ కట్టడం ----
bump(word, delta = 1) {
  const path = this.#path(word, false);
  if (!path) return { ok: false, reason: `NOT_FOUND: ${word}` };
  const leaf = path[path.length - 1];
  if (leaf.word === null) return { ok: false, reason: `NOT_A_WORD: ${word}` };
  leaf.count += delta;
  for (let i = path.length - 1; i >= 0; i--) this.#recompute(path[i]);
  return { ok: true, count: leaf.count };
}

remove(word) {
  const path = this.#path(word, false);
  if (!path) return { ok: false, reason: `NOT_FOUND: ${word}` };
  const leaf = path[path.length - 1];
  if (leaf.word === null) return { ok: false, reason: `NOT_A_WORD: ${word}` };
  leaf.word = null; leaf.count = 0;
  for (let i = path.length - 1; i >= 0; i--) this.#recompute(path[i]);
  // ఖాళీ ఆకులని కత్తిరించడం
  for (let i = path.length - 1; i > 0; i--) {
    const n = path[i];
    if (n.kids.size || n.word) break;
  }
}

```

```

    path[i-1].kids.delete(n.ch); this.#nodes--;
  }
  return { ok: true };
}

suggest(prefix, k = this.k) {
  let cur = this.#root;
  for (const ch of prefix) {
    cur = cur.kids.get(ch);
    if (!cur) return [];
  }
  return cur.top.slice(0, k).map(([w]) => w);
}
}

```

నిఘంటువు: 2,34,428 ప్రత్యేక పదాలు

```

loadAll      : 247 ms    · ఒక్కొక్కటిగా insert: 880 ms
suggest      : 0.2 μs/keystroke
nodes        : 7,58,880

```

```

"car" → car, card, care, careful, carry
"fa"  → father, family, face, fact, factory
"hel" → hello, help, helbeh, helcosis, helepole
"ind" → india, index, indian, ind, indagatory

```

```

helicopter trending → { ok: true, count: 9000003 }
"hel" ఇప్పుడు → helicopter, hello, help, helbeh, helcosis
"h"   ఇప్పుడు → helicopter, he, his, have, had

```

loadAll తర్వాత – 1,493 prefixes · సరైనది ✓
 1,998 bump/insert/remove తర్వాత – 1,494 prefixes · సరైనది ✓

మూడు కొలతలని పక్కపక్కన పెడితే

	ఒక్కొక్క keystroke ఖర్చు	సమయం
§7 · ranked scan	2,34,428 పదాలు	2.29 ms
§8 · plain trie	8,550 nodes (1-అక్షరం: 49,107)	0.46 ms
§13 · cached top-K	4.4 nodes	0.2 μs

2.29 ms → 0.2 μs. దాదాపు పదకొండు వేల రెట్లు – మరియు జవాబులు అక్షరాలా ఒకటే.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	<code>filter</code> + <code>slice</code>	మౌలిక అస్థిపంజరం
§4 (విరుపు)	<code>count</code> మరియు <code>sort</code>	"hel" → helbeh, helcoid

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§5	<code>localeCompare</code> tiebreak	సమాన ప్రజాదరణకి స్థిర క్రమం
§7 (విరుపు)	<code>Trie</code>	ఒక్కే keystroke కి 2.34 లక్షల పదాలు
§8	–	Trie కేవలం సగం పరిష్కారం
§9	<code>top[K]</code> ప్రతి node వద్దా	1-అక్షరం prefix కి 49,107 nodes
§10 (విరుపు)	<code>#recompute</code> దారి వెంట	Trending పదం కనిపించలేదు
§11	<code>loadAll</code> విడిగా	Incremental build 3.6× నెమ్మది
§12	Fuzz test (సరిచేసినది)	పరీక్ష తప్పు system ని పరీక్షించింది

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Trie ని వేగంగా దాటండి - నిజమైన లోతు top-K cache lo ఉంది

5 నిమి

Clarify - ర్యాంకింగ్ దేని ప్రకారం? ఎంత తరచుగా మారుతుంది?

8 నిమి

Scan ఎందుకు సరిపోదు · కలిత చెప్పండి

9 నిమి - Trie

node ని కనుక్కోవడం · సగం మాత్రమే

15 నిమి - top-K cache

ఎందుకు K చాలు · invalidation

8 నిమి

typos · వ్యక్తిగతీకరణ · scale

ఏమి తప్పక చెప్పాలి

1. "Top అంటే ఏమిటి?" అని అడగండి (§4) – మరియు `slice(0,5)` ఎందుకు తప్పో ఒక ఉదాహరణతో చెప్పండి. "hel → helbeh" అనేది 15 సెకన్లు మరియు అది గుర్తుండిపోతుంది.
2. Trie కేవలం సగం పరిష్కారం (§8) – ఇది చాలా మంది చెప్పరు. "Finding the node is O(prefix), but ranking still walks the subtree."
3. ప్రతి node వద్దా top-K దాచొచ్చు – మరియు ఎందుకో (§9) – "a word that's 6th in a child can never be in the parent's top-5." ఆ వాదన చెప్పడం ముఖ్యం, కేవలం "cache చేస్తాను" అనడం కాదు.
4. Cache invalidation ఒక దారి వెంట (§11) – మరియు bulk vs incremental build.
5. Tiebreak నిర్ణయాత్మకంగా ఉండాలి (§5) – లేకపోతే సూచనలు కారణం లేకుండా మారతాయి.

ఏమి వదిలేయాలి

- merge రాయొద్దు – "merge two sorted top-K lists, keep the best K" అని చెప్తే చాలు.
- remove మరియు pruning – అడిగితేనే.
- Memory optimisation (§16) – ప్రస్తావించండి, అమలు చేయొద్దు.
- Typo tolerance – ఒక వాక్యం, ఎందుకంటే అది వేరే structure.

15. నీటితో చెప్పాల్సిన English script

"Two questions before anything else. What does 'top' mean – top by what? And how often does that ranking change?

I ask the first one because it's the easiest thing to get silently wrong. The obvious implementation filters the word list by prefix and takes the first five. I ran that against a real dictionary – about two hundred and thirty-four thousand words – and typing 'hel' returns helbeh, helcoid, helcology, helcoplasty and helcosis. Those are real words; nobody has ever searched for them. The list is alphabetical, so 'first five' is always 'alphabetically earliest', which is almost always obscure. No crash, no failing test, completely unusable product. So suggestions have to be ranked by something – search volume, usually – with a deterministic tiebreak so they don't shuffle for no reason.

Once ranking is right, cost becomes the problem. Autocomplete runs on every keystroke, so typing one ten-letter word is ten searches. I measured fifteen hundred keystrokes: each one scanned all two hundred and thirty-four thousand words to return five. That's about two point three milliseconds per keystroke, which one user can live with – but a search box at ten thousand queries per second would need twenty-three seconds of CPU per second.

The structural fix is a trie. A prefix is a path, not a filter, so reaching the 'hel' node is three steps instead of a quarter-million comparisons. But I'd be explicit that this only solves half the problem: finding the node is cheap, and then ranking still means walking everything underneath it. I measured that too – a one-letter prefix still visited about forty-nine thousand nodes. And that's exactly the wrong shape, because the first keystroke is the most expensive and it's the one the user waits on.

So the real answer is caching the top K at every node. The reason that works is worth stating: if a word is sixth-best inside some child's subtree, five words in that same subtree beat it, and all five are also in the parent's subtree – so it can never be in the parent's top five. That means a node's top five is always derivable from its children's top fives, and each node needs to store only five entries. Building it is one bottom-up pass, and a suggestion becomes a walk down the prefix and a slice. That took two point three milliseconds down to about two tenths of a microsecond, with identical answers.

The catch is the one every cache has: when a word's popularity changes, the cached lists above it are stale. I had a word become the most-searched in the dictionary and it still didn't appear under its own prefix, because only its own node knew. The fix is cheap because the trie shape tells you exactly what's affected – only the nodes on the path from the root to that word can change, so you recompute that path bottom-up. Ten nodes, not three-quarters of a million.

One practical detail: that makes every insert do path work too, so the initial load gets about three and a half times slower. I'd keep two entry points – a bulk load that inserts everything and does one bottom-up pass at the end, and incremental insert and bump for a running system.

For confidence I'd fuzz it: thousands of random bumps, inserts and removes, checking every prefix against a naive sorted scan afterwards."

16. Follow-ups – అక్షర దోషాలు, వ్యక్తిగతీకరణ, scale

Follow-up	జవాబు	మారే classes
"K ని 10 చెయ్యి"	<code>new Autocomplete(10)</code>	0
"పదాలు కాదు, పూర్తి queries"	ఏమీ మారదు – "how to cook rice" కూడా ఒక key	0
"ప్రతి శోధనకీ count పెంచాలి"	<code>bump(query, 1)</code> ప్రతి శోధనకీ	0
"Memory ఎక్కువ (7.5 లక్షల nodes)"	కింద చూడండి	Node
"మధ్యలో సరిపోవాల్సి ('book' → 'facebook')"	Trie సరిపోదు – suffix automaton లేదా n-gram index	కొత్త structure
"అక్షర దోషాలు ('gogle')"	కింద చూడండి	కొత్త layer
"ఒక్కో user కి వేరే సూచనలు"	కింద చూడండి	కొత్త layer

Memory

"2,34,428 పదాలకి **7,58,880 nodes** – ఒక్కో పదానికి ~3.2 nodes. ప్రతి node lo ఒక `Map` మరియు ఐదు entries ఉన్నాయి, కాబట్టి ఇది చాలా పెద్దది.

రెండు ప్రామాణిక తగ్గింపులు:

- **Radix tree (compressed trie)** – ఒక్కో పిల్ల మాత్రమే ఉన్న గొలుసులని ఒకే node గా కుదించడం. "helcoplasty" lo "helco" తర్వాత ఒకే దారి ఉంటే, దాన్ని విడగొట్టడం వృథా.
- **Map కి బదులు 26-పొడవు array** (లేదా ఒక bitmask + packed array) – చిన్న alphabet కి అది చాలా తక్కువ memory.

మరియు ఒక ముఖ్యమైన ఎంపిక: top-K ని **అన్ని nodes వద్దా** దాచాల్సిన అవసరం లేదు. మొదటి 3-4 అక్షరాల nodes వద్ద మాత్రమే దాచి, లోతైన వాటికి subtree ని నడవోచ్చు – ఎందుకంటే అక్కడ subtree చిన్నది (98 పట్టిక: 4 అక్షరాలకి 529 nodes). **ఖర్చు ఎక్కడ ఉందో అక్కడే cache చేయడం.**"

అక్షర దోషాలు మరియు వ్యక్తిగతీకరణ

"**Typos** – trie ఒక ఖచ్చితమైన prefix నడక, కాబట్టి 'gogle' ఎక్కడికీ చేరదు. దీనికి **edit distance** కావాలి, మరియు సాధారణ పద్ధతి ఒక **BK-tree** లేదా trie మీద fuzzy నడక (ప్రతి అడుగులోనూ ఒక దోషం భరిస్తూ). అది ఖరీదు, కాబట్టి నిజమైన systems దీన్ని **ఖచ్చితమైన ఫలితాలు తక్కువగా ఉన్నప్పుడు మాత్రమే** నడుపుతాయి.

వ్యక్తిగతీకరణ – ఒక్కో user కి వేరే trie కట్టకూడదు (కోట్ల users). బదులుగా: **ఒక ఉమ్మడి trie + ఒక చిన్న వ్యక్తిగత జాబితా**, మరియు చూపించే ముందు రెండింటినీ కలపడం. వ్యక్తిగత జాబితా చిన్నది (ఆ user యొక్క గత శోధనలు), కాబట్టి దాన్ని scan చేయడం ఉచితం.

ఈ రెండూ ఒకే ఆకారం: ఖరీదైన సాధారణ నిర్మాణాన్ని అందరూ పంచుకుంటారు; వ్యక్తిగత/అరుదైన భాగం చివర్లో కలుపుతారు."

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
"Top" ని నిర్వచించకపోతే జవాబు అర్థరహితం	<code>slice(0,5)</code> (§4)	Search, feeds, recommendations
Prefix ఒక దారి, ఒక filter కాదు	Trie (§8)	Deep Dive 12 §5 · routing, IP lookup
సగం పరిష్కారాన్ని గుర్తించడం	Trie node ని కనుక్కుంది, ర్యాంక్ చేయలేదు (§8)	ప్రతి "సరైన structure" ఎంపిక
K చిన్నదైతే ప్రతి చోటా దాచొచ్చు	top-K merge (§9)	Deep Dive 13 · top-K ఎక్కడైనా
ప్రతి cache కి ఒక invalidation కథ	Trending పదం (§10)	Deep Dive 15 §16 · ప్రతి derived value
Bulk load ≠ incremental update	<code>loadAll</code> vs <code>insert</code> (§11)	Indexes, aggregates, caches
పరీక్ష తిరస్కరణలని గౌరవించాలి	<code>bump</code> return విస్మరించడం (§12)	ప్రతి fuzz test

ఒక చివరి ఆలోచన - ఈ DOC LO అత్యంత విలువైన క్షణం

§12 lo నా fuzz test విఫలమైంది, మరియు నేను వెంటనే నా **merge function** ని అనుమానించాను. దాన్ని మూడుసార్లు చదివాను. ఒక చిన్న పునరుత్పత్తి రాశాను - అది **pass** అయింది.

ఆపై ఖచ్చితమైన operation ని వెతికాను. Bug **code lo లేదు** - అది నా పరీక్షలో ఉంది.

పాఠం రెండు భాగాలు: ఒకటి, ఒక పరీక్ష విఫలమైతే పరీక్ష కూడా అనుమానితురాలే. రెండు - మరియు ఇది ముఖ్యమైనది - ఒక operation ఒక తిరస్కరణని తిరిగి ఇస్తే, దాన్ని విస్మరించే ఏ code ఐనా తప్పు, అది test ఐనా, production ఐనా.

ఈ series అంతటా "తిరస్కరణ ఒక API" అని చెప్తూ వచ్చాను. §12 lo నేనే దాన్ని విస్మరించాను.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard · 14 Text Editor · 15 Library · 16 Tic-Tac-Toe · 17 Autocomplete.**

Deep Dive 13 (Leaderboard) దీనికి దగ్గరి బంధువు - అక్కడా top-K, కానీ prefix లేకుండా, మరియు ranks కూడా కావాలి.

Autocomplete - అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి సంఖ్య నిజమైన 2,34,428-పదాల నిఘంటువుతో **node lo run** చేసి తీసినదే 